ICT1012

OPERATING SYSTEMS

LABORATORY MANUAL

Lab4-w10: Lab Assignments

AY 2025/26

BACHELOR OF ENGINEERING

IN

INFORMATION AND COMMUNICATIONS TECHNOLOGY

Contents

Lab Assignment 3
Prerequisite 3
Submission deadline 3
Assignment Task: "*mmap*" (memory map) and "*munmap*" (memory unmap) 3
Task introduction 3
Implementation 4
Testing the task 11
Run all test cases 12
Assignment submission 12

Lab Assignment

Prerequisite

- Download "**lab.zip**" from "xSiTe" folder "**Labs/xv6labs-w10: Mmap**" to "**c:/ICT1012/xv6labs-w10**".
- Unzip and keep all folders and files directly under "**c:/ICT1012/xv6labs-w10**" without the "labs" sub-folder.

Submission deadline

You must submit your result to Gradescope before 27 March 2026 11:59pm

Assignment Task: "*mmap*" (memory map) and "*munmap*" (memory unmap)

Task introduction

- The "*mmap*" and "*munmap*" system call allows UNIX programs to exert detailed control over their address spaces. They can be used to share memory among processes, to map files into process address spaces, and as part of user-level page fault schemes such as the garbage-collection algorithms discussed in lecture.
- This task is to add "*mmap*" and "*munmap*" to xv6, focusing on memory-mapped files.
- "**mmap**":
    - In UNIX, the manual page (run **man 2 mmap**) shows this declaration for "*mmap*":

```
mmap(2)                                          System Calls Manual

NAME
       mmap, munmap - map or unmap files or devices into memory

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/mman.h>

       void *mmap(void addr[.length], size_t length, int prot, int flags,
                  int fd, off_t offset);
       int munmap(void addr[.length], size_t length);
```

addr: assume always be zero, meaning that the kernel should decide the virtual address at which to map the file. "*mmap*" returns that address, or 0xffffffffffffffff if it fails.

len: the number of bytes to map; it might not be the same as the file's length.

prot: indicates whether the memory should be mapped readable, writeable, and/or executable; assume that "***prot***" is **PROT_READ** or **PROT_WRITE** or **both**.

flags: will be either **MAP_SHARED**, meaning that modifications to the mapped memory should be written back to the file, or **MAP_PRIVATE**, meaning that they should not. Not required to implement any other bits in flags.

fd: is the open file descriptor of the file to map.

offset: assume offset is zero (it's the starting point in the file at which to map).

- The "*mmap*" implementation should fill in the page table lazily, in response to page faults. That is, "*mmap*" itself should not allocate physical memory or read the file. Instead, do that in page fault handling code in (or called by) "***usertrap***". The reason to be lazy is to ensure that "*mmap*" of a large file is fast, and that "*mmap*" of a file larger than physical memory is possible.
- It's OK if processes that map the same MAP_SHARED file do not share physical pages.

- "**munmap**":
  - In UNIX, the manual page (run **man 2 mmap**) shows this declaration for "*munmap*":

```
mmap(2)                                          System Calls Manual

NAME
       mmap, munmap - map or unmap files or devices into memory

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/mman.h>

       void *mmap(void addr[.length], size_t length, int prot, int flags,
                  int fd, off_t offset);
       int munmap(void addr[.length], size_t length);
```

addr: address from which the memory -mapped file has to be unmapped.

len: the number of bytes to "*unmap*".

  - "*munmap*" should remove "*mmap*" mappings in the indicated address range, if any.
  - If the process has modified the memory and has it mapped MAP_SHARED, the modifications should first be written to the file.
  - An "*munmap*" call might cover only a portion of an "*mmap-ed*" region, but it can be assumed that it will either "*unmap*" at the start, or at the end, or the whole region (but not punch a hole in the middle of a region).
  - When a process exits, any modifications it has made to MAP_SHARED regions should be written to the relevant files, as if the process had called "*munmap*".
- Implementation of "*mmap*" and "*munmap*" functionality will be tested by user program "***mmaptest***" and should pass all tests. If "*mmaptest*" doesn't use a "*mmap*" feature, that feature need not be implemented.

Implementation

"***mmaptest***":

- This is the user level program to test the implementation of "*mmap*" and *munmap*".

- "*_mmaptest*", the executable is already been added to UPROGS in "*Makefile*".
- Summary of tests carried out by the "*mmaptest*" are:

Test Name	Description / What it Tests	Expected Kernel Behavior
Basic mmap	Verifies the simplest mapping of a file into the process's address space.	The kernel must map file contents into virtual memory and allow reading.
mmap private	Tests MAP_PRIVATE functionality where modifications are local to the process.	Changes to memory must not be written back to the underlying file.
mmap read-only	Tests permission enforcement when mapping a read-only file with write permissions.	The mmap call must fail if a read-only file is requested as PROT_WRITE | MAP_SHARED.
mmap read/write	Tests MAP_SHARED functionality where modifications are shared.	The kernel must allow mapping and writing to a file opened with O_RDWR.
mmap dirty	Verifies that modified "dirty" pages are correctly synchronized to the disk.	When "*munmap*" is called on shared memory, changes must appear in the actual file.
not-mapped unmap	Tests the ability to unmap portions of a mapping or memory that isn't fully mapped.	The kernel must correctly track the remaining valid portions of a VMA.
lazy access	Verifies that the file is not read into memory until the page is actually accessed.	The mapping should be "lazy"; file reads should happen via page faults, not at "*mmap*" time.
mmap two files	Tests the kernel's ability to maintain multiple distinct VMAs simultaneously.	Mapping a second file must not interfere with or overwrite the first file's mapping.
fork test	Tests if memory mappings are correctly duplicated and managed across "*fork()*".	Child processes must inherit VMAs and see the same mapped file content.
munmap prevents access	Verifies that memory becomes inaccessible immediately after being unmapped.	Any access to an unmapped address must cause a fatal fault and kill the process.
writes to read-only	Tests hardware-level protection for read-only memory mappings.	Attempting to write to a PROT_READ mapping must trigger a page fault and kill the process.

Step1: Define the structure of virtual memory area (VMA)

- Define VMA in "*kernel/proc.h*":
  - Define a structure corresponding to the VMA (virtual memory area). This should record the address, length, permissions, file, etc. for a virtual memory range created by "*mmap*".

```
// define the structure of "virtual memory area (VMA)"
struct vma {
  int valid;                // Is this VMA slot in use?
  uint64 addr;              // Starting virtual address
  int length;               // Size of the mapping
  int prot;                 // Permissions (PROT_READ, PROT_WRITE)
  int flags;                // MAP_SHARED or MAP_PRIVATE
  struct file *f;           // Pointer to the mapped file
  int offset;               // Offset within the file
};
```

  - Since the xv6 kernel doesn't have a variable-size memory allocator in the kernel, declare a fixed-size array of VMAs and allocate from that array as needed. A size of 16 should be sufficient.

```
#define MAX_VMA 16
```

  - Add "*struct vma*" to "*struct proc*"

```
// Per-process state
struct proc {
  struct spinlock lock;

  // … existing member variables …

  // ICT1012 Lab 4 ----------------
    struct vma vmas[MAX_VMA]; // Maximum of 16 concurrent mappings per process
//-------------------------------
};
```

- Define protection options and flags in "*kernel/fcntl.h*":
  - Refer the '*prot*' argument of "*mmap*" function. This argument describes the desired memory protection of the mapping (and must not conflict with the open mode of the file). Add "*PROT_READ*" and "*PROT_WRITE*".

```
#define PROT_READ 0x1       // Pages may be read
#define PROT_WRITE 0x2      // Pages may be written
```

  - Refer the "*flags*" argument of "*mmap*" function. The flags argument determines whether updates to the mapping are visible to other processes mapping the same region, and whether updates are carried through to the underlying file. Add "*MAP_SHARED*" and "*MAP_PRIVATE*"

```
#define MAP_SHARED 0x01     // Share this mapping
#define MAP_PRIVATE 0x02    // Create a private copy-on-write mapping
```

- "*kernel/riscv.h*": Add the "PTE_A" and "PTE_D" Page Table Entry (PTE) status bits

- PTE_A (Accessed bit): The hardware sets this bit to 1 whenever the page is read, written, or executed. It helps the Operating System implement Page Replacement Algorithms (like Least Recently Used). By periodically checking and clearing this bit, the kernel can tell which pages are frequently used and which can be swapped to disk.
- 2. PTE_D (Dirty bit): The hardware sets this bit to 1 only when the page is written to. This is critical for the "*mmap*" lab. It tells the kernel if the version of the page in RAM is "dirty" (different from the version on the disk).

```
#define PTE_A (1L << 6) // Accessed
#define PTE_D (1L << 7) // Dirty
```

- Update "*allocproc()*" function in "*kernel/proc.c*" to initialize the "*vma*" structure when a process is allocated.

```
...
p->vmas[i].valid = 0;
...
```

Step2: Register the two system calls: "*mmap*" and "*munmap*"

- "*kernel/syscall.h*": Add two new system calls "*SYS_mmap*" and "*SYS_munmap*" and assign new system call numbers to them.
- "*kernel/syscall.c*": Register the system calls.

```
static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
...
[SYS_mmap]    sys_mmap,
[SYS_munmap]  sys_munmap,
};
```

```
extern uint64 sys_mmap(void);
extern uint64 sys_munmap(void);
```

- "*user/user.h*": Add "*mmap*" and "*munmap*" prototypes

```
void *mmap(void *, uint, int, int, int, uint);
int munmap(void *, uint);
```

- "*user/usys.pl*": Add the following

```
entry("mmap");
entry("munmap");
```

- "*kernel/sysfile.c*": Add "*sys_mmap*" and "*sys_munmap*" system call functions. For now, just return errors from "*mmap*" and "*munmap*".

```
uint64
sys_mmap(void)
{

  return -1;
}
```

```
uint64
sys_munmap(void)
{

  return -1;
}
```

- Make sure that by this stage the whole program is able to compile.

Step3: Implement the "*sys_mmap*" system call handler

- Update "*sys_mmap*" function in "*kernel/sysfile.c*":
  - Argument Extraction & Validation:
    - Fetch all six arguments (addr, length, prot, flags, fd, offset) using argaddr, argint, and argfd.
    - Permissions Check: If the file is mapped with MAP_SHARED and PROT_WRITE, ensure the underlying file is opened for writing (f->writable).
    - Basic Constraints: Ensure the file is at least readable. In this lab, you may assume offset is always 0 and addr is a hint you can ignore.
    - Alignment: Use PGROUNDUP on the requested length to ensure the mapping occupies full pages.
  - VMA Allocation:
    - Find an unused slot in the process's Virtual Memory Area (VMA) space (p->vmas) and add a VMA to the process's table of mapped regions. The VMA should contain a pointer to a struct file for the file being mapped. If no slots are available, return an error.
  - Address Space Placement (The "Where"):
    - The kernel must find a region in the virtual address space that doesn't overlap with the heap or other VMAs.
    - Calculate a safe base address. A common strategy is to place VMAs in high memory (e.g., starting at 0x40000000) and iterate through existing VMAs to find the next available gap.
  - VMA Initialization:
    - Store the calculated addr, length, prot, flags, and offset into the chosen VMA slot.
    - Mark the slot as valid.
  - File Reference Management:
    - Since the VMA now holds a pointer to the struct file, you must increment the file's reference count using "filedup(f)". This prevents the file structure from being freed if the user calls "close(fd)" while the mapping still exists.
  - Return the Virtual Address:
    - Return the virtual address where the mapping starts.
- Run "*mmaptest*": the first "*mmap*" should succeed, but the first access to the "*mmap-ed*" memory will cause a page fault and kill "*mmaptest*".

Step4: Handle page faults

- When the process accesses the mapped memory, it will trigger a page fault. Update "*usertrap*" function in "*kernel/trap.c*" to catch the page fault and to handle it.
- When a process accesses a VMA address that hasn't been loaded into RAM yet, the hardware triggers a page fault, and your code must decide whether to load the data or kill the process.

  1. Identify the Fault Type: In RISC-V, page faults are identified by the "*scause*" register

     - "*scause*" 13: Load Page Fault (the process tried to read memory).
     - "*scause*" 15: Store Page Fault (the process tried to write to memory).

  2. Follow these steps within the section marked // ICT1012 Lab 4 ----------:

     1. Get the Faulting Address: Retrieve the virtual address that caused the fault using "*r_stval()*".
     2. Find the Matching VMA: Iterate through the current process's VMA array (p->vmas). Check if the faulting address falls within the range of any valid VMA.
     3. Handle VMA Access:
        - If a VMA is found:
          - Check Permissions: If the fault is a Store Page Fault (15) but the VMA is read-only (!(v->prot & PROT_WRITE)), the access is illegal. Print an error and kill the process.
          - Load the Data: If permissions are correct, call the provided "*mmap_handler(va, v)*" function to lazily load the file content into a physical page. If it returns an error, kill the process.
        - If NO VMA is found:
          - Check for Lazy Heap: Check if the address is below the process heap limit (p->sz) and above the stack pointer. If it is, call "*vmfault()*" to handle standard lazy allocation.
          - Final Fallback: If the address belongs to neither a VMA nor the heap, it is an invalid memory access. Kill the process.

- Run "*mmaptest*"; it should get to the first "*munmap*".

Step5: Implement the "*sys_munmap*" system call handler

- Update "*sys_munmap*" function in "*kernel/sysfile.c*":

The "*munmap*" system call removes a memory mapping. The goal is to identify the correct VMA, use the provided helper to clean up the memory/disk state, and then update the process's VMA metadata.

- Extract Arguments: Use "argaddr" and "argint" to get the target address and length.
- Find the VMA: Iterate through VMAs. Find the VMA that contains the requested range.

  Constraint: For this lab, you can assume the unmap range will always be at the very beginning, the very end, or cover the entire VMA.

- Perform the Unmap: Call the provided helper function "*vma_unmap(p, v, addr, length)*" in "*kernel/proc.c*"

  This function checks for modified (dirty) pages, writes them back to the file if the mapping is MAP_SHARED, and then removes the pages from the page table using "*uvmunmap*".

- Update VMA Metadata (Crucial):

  After "*vma_unmap*" handles the memory and disk synchronization, you must update the VMA structure to reflect the new boundaries. Since the lab only requires unmapping from the edges, handle these three cases:

  - Full Unmap (The range matches the VMA exactly):

    Close the file reference (*fileclose*) and mark the VMA as invalid.

  - Partial Unmap from Start (Unmapping from the base address upwards):

    Shift the VMA's start address (addr) and file offset forward by the unmapped length. Decrement the VMA's length.

  - Partial Unmap from End: (Unmapping from the top downwards)

    Simply decrement the VMA's length.

- Clean up: Call "sfence_vma()" to flush the TLB (Translation Lookaside Buffer).

Step6: Modify the "*kfork()*" function in "kernel/proc.c"

- Ensure that the child has the same mapped regions as the parent.
- Increment the reference count for a VMA's struct file.

```
…
// 1. Copy the VMA metadata from parent
np->vmas[i] = p->vmas[i];

// 2. Increment the file reference count
// The child now also "owns" a reference to this mapped file
```

```
filedup(p->vmas[i].f);
…
```

Step7: Modify the "*kexit()*" function in "kernel/proc.c" *to invalidate the VMA slot*

- When a process terminates, it must clean its VMA.
- If a page in VMA is dirty, the content of the page needs to be written back to mapped file.
- For each page in VMA, if it is in use (valid):
    1. Sync and unmap the memory range
    2. Decrement the file reference count
    3. Invalidate the VMA slot

```
…
// 1. Sync and unmap the memory range
vma_unmap(p, &p->vmas[i], p->vmas[i].addr, p->vmas[i].length);

// 2. Decrement the file reference count
fileclose(p->vmas[i].f);

// 3. Invalidate the VMA slot
p->vmas[i].valid = 0;
…
```

- Run "*mmaptest*"; it should pass all the tests.

Testing the task

Run "**make clean && make qemu**". Then run "**mmaptest**"

```
$ make clean && make qemu
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
test basic mmap
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
```

```
test munmap prevents access
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
            sepc=0xbfa stval=0x40004000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded
$
```

Run all test cases

Run **"./grade-lab-mmap mmaptest"**

```
$ ./grade-lab-mmap mmaptest
make: 'kernel/kernel' is up to date.
== Test running mmaptest == (2.6s)
== Test   mmaptest: mmap basic ==
  mmaptest: mmap basic: OK
== Test   mmaptest: mmap private ==
  mmaptest: mmap private: OK
== Test   mmaptest: mmap read-only ==
  mmaptest: mmap read-only: OK
== Test   mmaptest: mmap read/write ==
  mmaptest: mmap read/write: OK
== Test   mmaptest: mmap dirty ==
  mmaptest: mmap dirty: OK
== Test   mmaptest: not-mapped unmap ==
  mmaptest: not-mapped unmap: OK
== Test   mmaptest: lazy access ==
  mmaptest: lazy access: OK
== Test   mmaptest: two files ==
  mmaptest: two files: OK
== Test   mmaptest: fork_test ==
  mmaptest: fork_test: OK
== Test   mmaptest: munmap_noaccess ==
  mmaptest: munmap_noaccess: OK
== Test   mmaptest: read_only_write ==
  mmaptest: read_only_write: OK
$
```

Assignment submission

1. Execute "**make clean && make grade**" to test "*mmap*" and "*munmap*" tasks.

```
$ make clean && make grade
make[1]: Leaving directory '/mnt/c/ICT1012/xv6labs-
w10_instructor'
== Test running mmaptest ==
$ make qemu-gdb
(7.3s)
== Test   mmaptest: mmap basic ==
  mmaptest: mmap basic: OK
```

```
== Test   mmaptest: mmap private ==
  mmaptest: mmap private: OK
== Test   mmaptest: mmap read-only ==
  mmaptest: mmap read-only: OK
== Test   mmaptest: mmap read/write ==
  mmaptest: mmap read/write: OK
== Test   mmaptest: mmap dirty ==
  mmaptest: mmap dirty: OK
== Test   mmaptest: not-mapped unmap ==
  mmaptest: not-mapped unmap: OK
== Test   mmaptest: lazy access ==
  mmaptest: lazy access: OK
== Test   mmaptest: two files ==
  mmaptest: two files: OK
== Test   mmaptest: fork_test ==
  mmaptest: fork_test: OK
== Test   mmaptest: munmap_noaccess ==
  mmaptest: munmap_noaccess: OK
== Test   mmaptest: read_only_write ==
  mmaptest: read_only_write: OK
Score: 150/150
$
```

2. Execute "**make zipball**" to create "**lab.zip**".

3. Upload the "**lab.zip**" file to Gradescope assignment "**xv6labs-w10: mmap**" to get auto graded.